M03-L01 · Tres agentes, tres sospechosos

Alberto Basalo

TESTING INTELIGENTE

Triage de fallos

> *Dos agentes construyen la funcionalidad y la prueba. Otro la ejecuta y ve qué falla.*

CONNECTIONS

- Si el agente que escribe el código también escribe sus pruebas, ¿**quién** pone en duda sus supuestos?
- Las pruebas de la funcionalidad nueva pasan. ¿Has terminado, o falta **ejecutar toda la suite**?

CONCEPTS

Tres orígenes del fallo

Una prueba falla por:

- un error en **la prueba**
- en **el código** de producción
- o en **la spec**.

> *El mensaje de error muestra el síntoma, no el origen.*

CONCEPTS

Un agente para cada origen

Origen	Skills
spec	`/define-spec`, `/define-e2e-features`
código	`/implement-spec`
pruebas	`/implement-e2e-features`, `/review-e2e-tests`

Cada agente empieza de cero en su conversación.

No copia los supuestos de los otros ni es **indulgente** con lo ya hecho.

Quien detecta un fallo no lo corrige.

> *Lo decidimos por **independencia**, no por capacidad.*

CONCEPTS

La spec nueva y toda la suite

Las etiquetas permiten seleccionar un requerimiento, una spec o toda la suite.

- Ejecutar solo la spec nueva: es **rápido** mientras desarrollas.
- La suite completa es un comprobación **regresiva** de que no se rompe lo que funcionaba.

CONCRETE PRACTICE

La petición: reservas de plazas

> *Ahora los lanzamientos no llevan pasajeros. Queremos empezar a registrarlos.*
>
> *Un operador registra a cada pasajero en un lanzamiento. Los pasajeros no se registran solos.*
>
> *De cada pasajero necesitamos sus datos de contacto: nombre, nacionalidad, documento de identidad, email y teléfono.*
>
> *Cada lanzamiento tiene tantas plazas como la capacidad de su cohete. Antes de guardar una reserva hay que comprobar que queda plaza: no queremos overbooking.*
>
> *Un pasajero no puede tener dos reservas en el mismo lanzamiento.*
>
> *Solo se puede reservar en lanzamientos planificados o confirmados.*
>
> *El pago no entra ahora; lo haremos más adelante.*

CONCRETE PRACTICE

P1 · La conversación «spec»

Abre una conversación nueva. Usa `/define-spec` con la petición para crear la spec `S0005`. Después usa `/define-e2e-features` para escribir un `.feature` por historia.

Comprueba que cada criterio tiene su escenario, incluida la reserva duplicada.

Fallo provocado: quita de `spec.md` todo lo que dice de la reserva duplicada: la regla de negocio, el 409 de la solución, la nota de E2E y el criterio `R6`. No cambies los números de los otros criterios. Deja el `.feature` como está. Ahora la spec y el `.feature` no dicen lo mismo.

CONCRETE PRACTICE

P2 · La conversación «código»

Abre otra conversación nueva. Usa `/implement-spec` con `spec.md` de `S0005`. Este agente no lee el `.feature`.

Fallo provocado: en la comprobación de plazas, cambia `>=` por `>`. Ahora el sistema acepta una reserva de más.

La prueba unitaria del agente detecta el cambio. Cámbiala también: que acepte una reserva de más. Así simulas un agente que comete el mismo error en el código y en su prueba. `npm test` pasa.

CONCRETE PRACTICE

P3 · La conversación «pruebas»

Abre una tercera conversación nueva. Usa `/implement-e2e-features` con los `.feature` de `S0005`. Reutiliza los Page Objects y los datos existentes.

Este agente no sabe nada de los dos fallos provocados. Escribe las pruebas a partir de los escenarios.

No ejecuta las pruebas: solo comprueba los tipos y la lista. Quien escribe las pruebas no juzga su resultado.

CONCRETE PRACTICE

P4 · Ejecutar la spec nueva

Ejecuta tú, en la terminal y desde `e2e/`, solo las pruebas de `S0005`:

```
npx playwright test --grep '@S0005'
```

Fallan dos pruebas: `S0005-R6`, la reserva duplicada, y `S0005-R7`, el límite de plazas. Las dos esperan `409` y reciben `201`. Lee el resumen de la consola: el nombre de la prueba, sus etiquetas y el error.

CONCRETE PRACTICE

P4 · Ejecutar toda la suite

Ejecuta ahora todas las pruebas:

```
npx playwright test
```

Aparece un tercer fallo: `S0003-R7 (US2)`, que lista los lanzamientos. Nadie ha tocado esa prueba. La ejecución filtrada no podía verlo.

Solo anota los tres fallos. Leer sus pistas es el trabajo de L02.

CONCRETE PRACTICE

Los tres fallos abiertos

Fallo	Síntoma	Se resuelve en
`S0003-R7 (US2)`	La fila del lanzamiento tiene más texto del esperado	L02
`S0005-R6` · Reserva duplicada	Espera `409`, recibe `201`	L03
`S0005-R7` · Límite de plazas	Espera `409`, recibe `201`	L04

> *Cuidado: `S0003-R7` y `S0005-R7` son pruebas distintas. Usa siempre el código completo.*

Ya sabes qué fallos provocaste. Lo importante es cómo encuentran las pistas su origen y

CONCLUSION

Tres agentes, tres responsables

Cada agente trabaja de cero y responde de una parte. Cuando algo falla, sabes a quién devolver el fallo.

Antes de cerrar una funcionalidad

¿Has ejecutado **la spec nueva y toda la suite**?

Siguiente paso: leer las pistas

En L02 leerás las pistas de `S0003-R7 (US2)` para personas y para agentes, y otro agente corregirá la prueba.

Alberto Basalo